

Praktikumsbericht zu Versuch 8

Von Daniel Baer (3235621)

1. Dijkstra – Reindeer Maze

a) Modellierung

Für eine Karte ohne Wände mit R Zeilen und C Spalten gilt:

- Knotenanzahl = Richtungen * Zeilenzahl * Spaltenzahl = $4 * R * C$
- Vorwärtskanten = Drehungen * Zeilenzahl * Spaltenzahl = $2 * R * C$
- Drehkanten = $8 * R * C$

Zur Darstellung eignet sich eine Adjazenzliste, da weniger Speicher als bei einer Adjazenzmatrix benötigt wird.

b) Dijkstra

Kosten minimaler Pfad: 7036

Aktionsfolge:

LFFRFFFFLFFFFRFFFFRFFFFRFFFFLFFLFFFFFFFFFFF

Pfad:

```

13,1,O -> 13,1,N -> 12,1,N -> 11,1,N -> 11,1,O ->
11,2,O -> 11,3,O -> 11,4,O -> 11,5,O -> 11,5,N ->
10,5,N -> 9,5,N -> 8,5,N -> 7,5,N -> 7,5,O ->
7,6,O -> 7,7,O -> 7,8,O -> 7,9,O -> 7,10,O ->
7,11,O -> 7,11,S -> 8,11,S -> 9,11,S -> 10,11,S ->
11,11,S -> 12,11,S -> 13,11,S -> 13,11,O -> 13,12,O ->
13,13,O -> 13,13,N -> 12,13,N -> 11,13,N -> 10,13,N ->
9,13,N -> 8,13,N -> 7,13,N -> 6,13,N -> 5,13,N ->
4,13,N -> 3,13,N -> 2,13,N -> 1,13,N

```

c) AoC

Kosten minimaler Pfad: 133584

Aktionsfolge:

LFFFFFFRFFFFLFFLFFFFRFFFFRFFRFFLFFLFFFFLFFFFRFFFFRFFRFFLFFLFFFFLFFFFRFFRFFF
FLFFFFRFFFFLFFRFFRFFFFFFFFFFLFFLFFRFFRFFLFFRFFLFFRFFFFFFFFFFLFFFFFFFFFFF
FFFFFFFFFFFFFFFFRFFFFFFFFFFLFFFFFFFFFFFRFFLFFLFFFFFFFFFFFRFFRFFLFFLFFRFFR
FFLFFFFRFFFFFFFFFFLFFFLFFLFFRFFRFFFFFFFFFFFRFFLFFLFFFFRFF
FFFFFFFFRFFFLFFFFRFFLFFFFFFFFFFLFFRFFRFFLFFLFFFFFRFFLFFLFFRFFRFFLFFFFF
FFFFFFFFLFFFFRFFFFLFFRFFFFLFFRFFRFFLFFRFFRFFLFFRFFRFFLFFRFFRFFRFFRFFRFF
FFFRFFFFLFFFFLFFFFFRFFRFFFFLFFRFFLFFFFFFFFFFLFFRFFLFFRFFLFFRFFRFFFFFL
FFFFFFFFFFFFFFFFLFFLFFFFFFFFFFFRFFRFFLFFFFFFFFFFFRFFRFFLFFR
FFFRFFFFFFFFFFFFFFFFLFFRFFLFFLFFRFFLFFRFFLFFFFFLFFLFFRFFRFFLFFRFFRFFRFFLFF
FFLFFRFFRFFFFFFFFFFLFFRFFLFFFFRFFRFFLFFFFLFF

2. Bellman-Ford

a) Implementierung

```
def bellman_ford(graph, start_name: str) -> tuple[dict, dict, bool]:
    vertices = graph.all_vertices()
    distance_map = {vertex: float("inf") for vertex in vertices}
    predecessor_map = {vertex: None for vertex in vertices}

    start_vertex = graph.get_vertex(start_name)
    distance_map[start_vertex] = 0

    vertex_count = len(vertices)

    for _ in range(vertex_count - 1):
        changed = False
        for src_name, dest_name, weight in graph.all_edges():
            src_vertex = graph.get_vertex(src_name)
            dest_vertex = graph.get_vertex(dest_name)

            if distance_map[src_vertex] == float("inf"):
                continue

            candidate = distance_map[src_vertex] + weight
            if candidate < distance_map[dest_vertex]:
                distance_map[dest_vertex] = candidate
                predecessor_map[dest_vertex] = src_vertex
                changed = True

        if not changed:
            break

    has_negative_cycle = False
    for src_name, dest_name, weight in graph.all_edges():
        src_vertex = graph.get_vertex(src_name)
        dest_vertex = graph.get_vertex(dest_name)

        if distance_map[src_vertex] == float("inf"):
            continue

        if distance_map[src_vertex] + weight < distance_map[dest_vertex]:
            has_negative_cycle = True
            break

    return distance_map, predecessor_map, has_negative_cycle
```

b) Test

Start

- a: Distanz = 0, Pfad = a
- b: Distanz = 7, Pfad = a -> d -> c -> b
- c: Distanz = 2, Pfad = a -> d -> c
- d: Distanz = 4, Pfad = a -> d
- e: Distanz = 6, Pfad = a -> d -> e

c) Negativer Zyklus

has_negative_cycle gibt **true** zurück.

Zyklus wird erkannt durch: $\text{Distanz}[\text{Quelle}] + \text{Gewicht} < \text{Distanz}[\text{Ziel}]$

Zyklus: b -> d -> e -> b

Gewicht: $2 + 2 + (-8) = -4$

d) Komplexität

Dijkstra bei positiven Kanten.

Bellman-Ford bei negativen Kanten.